

AI 100 講 — 補充單元

Google Apps Script

最低門檻的自動化入口

不需要伺服器、不需要域名、只需要 Google 帳號

按 → 或空白鍵開始

今天的旅程

1

為什麼要學 GAS ？

最低門檻的自動化工具

2

能做什麼？

Sheets、Gmail、Drive、Calendar 全自動化

3

核心概念

容器綁定、觸發器、執行限制

4

實戰範例

每日報表 & Telegram Bot

5

GAS 在 AI 時代的角色

從手寫到 AI 自動開發

Part 1

為什麼要學 GAS ?

四個零門檻

你每天的重複操作



不需要伺服器

跑在 Google 的雲端上

不需要域名

Google 幫你搞定

不需要安裝

瀏覽器就能寫

只需 Google 帳號

每個人都有

GAS 能自動化什麼？

服務	自動化範例
Sheets	定時匯總數據、自動產生報表、資料清洗
Gmail	自動回覆、批次轉寄、郵件分類
Drive	檔案自動歸檔、權限批次管理、備份
Calendar	自動建立會議、衝突偵測、提醒通知
Forms	表單提交後自動處理、寄確認信
Docs / Slides	自動產生文件/簡報、批次更新內容

還有更多

用 `UrlFetchApp` 可以呼叫任何外部 API — Telegram Bot、LINE Bot、Slack、Notion 都行。

Part 2

核心概念

容器綁定、觸發器、限制

容器綁定 vs 獨立專案

容器綁定 (Container-bound)

Sheets → 新增 → Apps Script

新增 Sheets

專案綁定到容器

獨立專案 (Standalone)

script.google.com → 新增

專案

獨立專案

觸發器 (Trigger)

類型	說明	範例
時間觸發	定時執行	每天 8:00 寄報表
表單觸發	提交時觸發	填完表單自動寄確認信
編輯觸發	修改時觸發	Sheets 被改時自動驗證
開啟觸發	打開文件觸發	開 Sheets 時更新資料

執行限制

限制	免費帳號	Workspace 帳號
單次執行時間	6 分鐘	30 分鐘
每日觸發器	20 個	20 個
每日 Email	100 封	1,500 封
每日 UrlFetch	20,000 次	100,000 次

免費夠用
一般個人使用，免費帳號的額度已經非常充足。

Part 3

實戰範例

每日報表 & Telegram Bot

範例一：每日自動寄報表

```
function sendDailyReport() {  
  // 取得 Sheets 表格  
  const sheet = SpreadsheetApp.getActive().getSheetByName("報表");  
  const data = sheet.getDataRange().getValues();  
  
  // 計算總和  
  const total = data.slice(1).reduce((sum, row) => sum + row[2],  
    0);  
  
  // 寄 Email  
  GmailApp.sendEmail("boss@company.com",  
    "每日報表",  
    `報表數據: ${total} 元`);  
}  
// 設定每日 08:00 自動執行
```

範例二：Telegram Bot Webhook

```
function doPost(e) {  
  const data = JSON.parse(e.postData.contents);  
  const chatId = data.message.chat.id;  
  const text = data.message.text;  
  
  // 發送訊息  
  UrlFetchApp.fetch(  
    `https://api.telegram.org/bot${TOKEN}/sendMessage`,  
    { method: "post", payload: { chat_id: chatId, text: `👋: ${text}` } }  
  );  
}
```

免費的 Telegram Bot 後端 — 不需要租伺服器，GAS 就是你的 webhook 處理器。

GAS 的限制（也是為什麼需要 clasp）

限制	影響
網頁編輯器功能陽春	沒有 Copilot、沒有自動補全
沒有版本控制	改壞了回不去
無法多人協作	程式碼衝突沒辦法 merge
只支援 JavaScript	不支援 TypeScript
偵錯工具有限	console.log 要到 Execution log 看

解決方案

clasp — 把 GAS 搬回 VS Code 本地開發，完整 Git + TypeScript + Copilot 支援。（見 S16）

GAS 在 AI 時代的角色

- 1 Level 1：手動寫 GAS**
在網頁編輯器裡一行一行寫
- 2 Level 2：AI 幫你寫 GAS**
ChatGPT 生成 → 複製貼上到編輯器
- 3 Level 3：clasp + Claude Code**
AI 直接在本地開發 + 部署到 Google
- 4 Level 4：gws MCP + AI Agent**
AI 自主操控 Google 服務，不再需要寫腳本

GAS 是 Google 自動化的**地基**，後面的 clasp、gcloud、gws 都是在這個地基上蓋更高的樓。

一句話記住這堂課

每個人都有一個
免費的自動化引擎

它叫 **Google Apps Script**

不需要伺服器、不需要域名、只需要一個 Google 帳號

developers.google.com/apps-script